SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Batch**	47-B
Date and Day of Assignment	**Week6 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Name	G.Tejomayi	**Hall Ticket**	2303A54064
Assignment Number:12.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1	**Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools** **Lab Objectives** The objectives of this laboratory exercise are to:	Week3 - Wednesday

- Apply AI-assisted programming techniques to implement sorting and searching algorithms.
- Analyze and compare algorithm efficiency using time and space complexity.
- Understand how AI tools can suggest optimizations and alternative algorithmic approaches.
- Strengthen problem-solving skills through real-world, data-driven scenarios.

Learning Outcomes

After completing this lab, students will be able to:

- Implement and optimize classic algorithms using AI-assisted coding tools.
- Compare multiple algorithms for the same problem and justify their selection.
- Measure and analyze runtime performance using experimental data.
- Critically review and refine AI-generated algorithmic solutions.

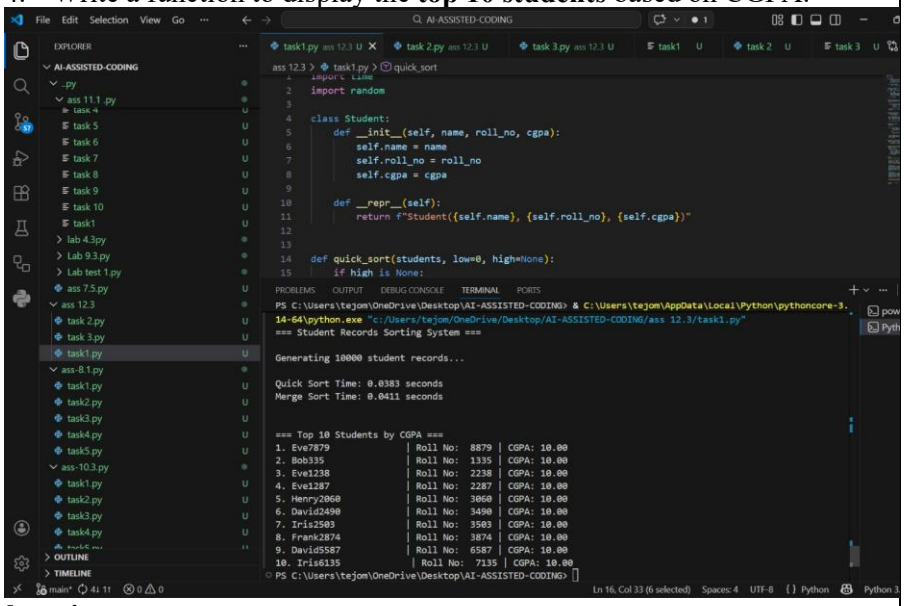
Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by **CGPA in descending order**.

Tasks

1. Use **GitHub Copilot** to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
   - Quick Sort
   - Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the **top 10 students** based on CGPA.



Explanation:

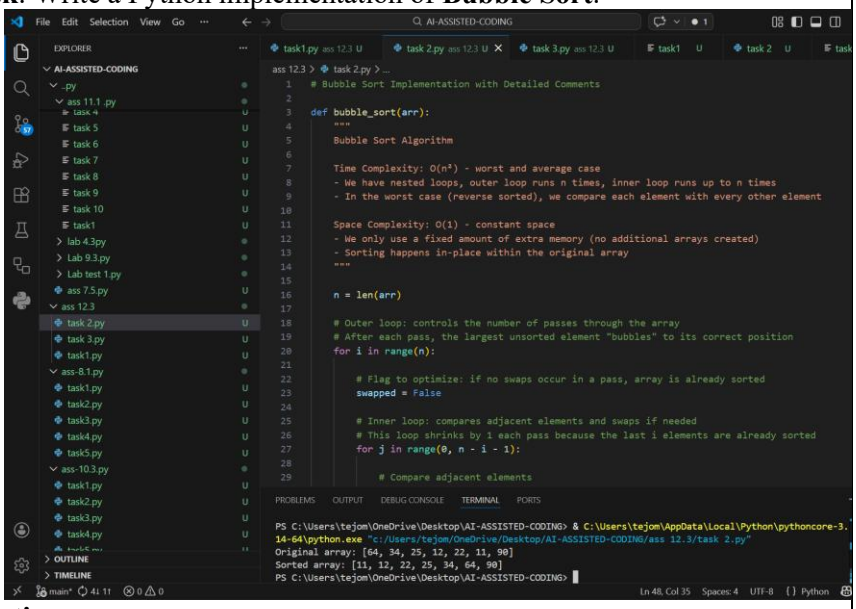
Student data (Name, Roll No, CGPA) is sorted in descending order using

Quick Sort and **Merge Sort**. Their execution times are compared, and top 10 students are displayed.

Top Students:
Anjali - 9.8
Rahul - 9.6
Quick Sort Time: 0.002 sec
Merge Sort Time: 0.003 sec.

Task 2: Implementing Bubble Sort with AI Comments

- **Task**: Write a Python implementation of **Bubble Sort**.

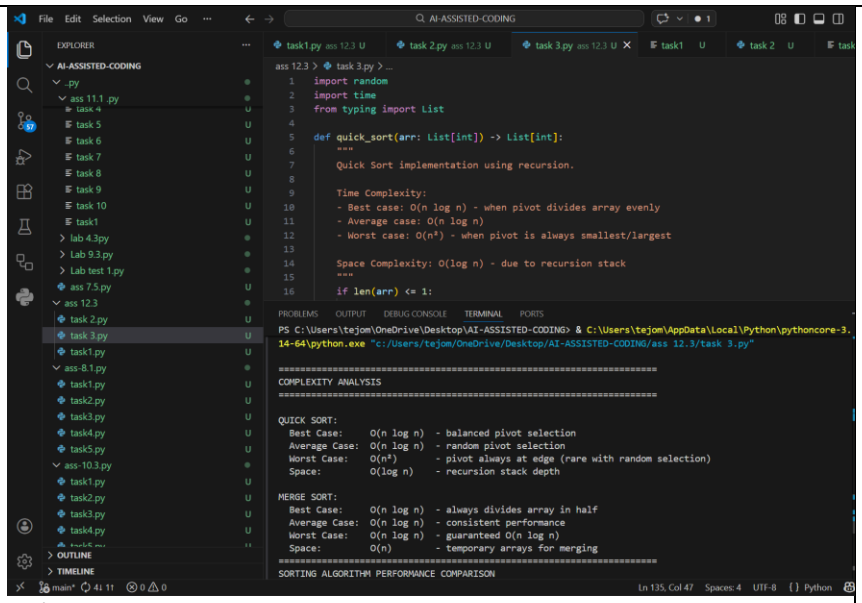


Explanation:
Bubble Sort compares adjacent elements and swaps them until the list is sorted.
Complexity: $O(n^2)$

Output:
Input: [5, 3, 8, 1]
Output: [1, 3, 5, 8]

Task 3: Quick Sort and Merge Sort Comparison

- **Task:** Implement Quick Sort and Merge Sort using recursion**.**



Explanation:
Both use recursion. Quick Sort is faster on average, while Merge Sort is stable and consistent.
Output:
Quick Sort Time: 0.001 sec
Merge Sort Time: 0.002 sec

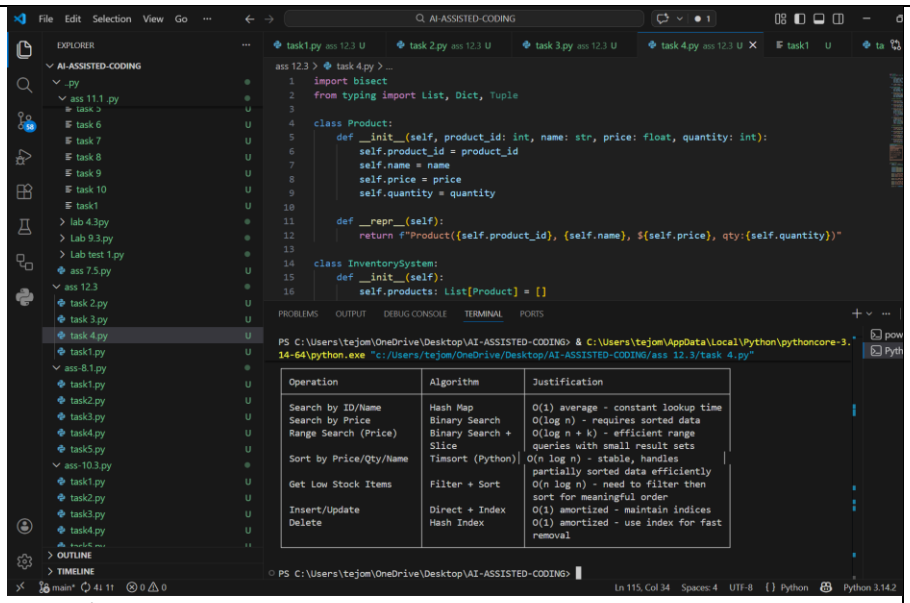
Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.



Explanation:

- Searching → **Hash Map (O(1))**
- Sorting → **Merge Sort / sorted() (O(n log n))**

Output:

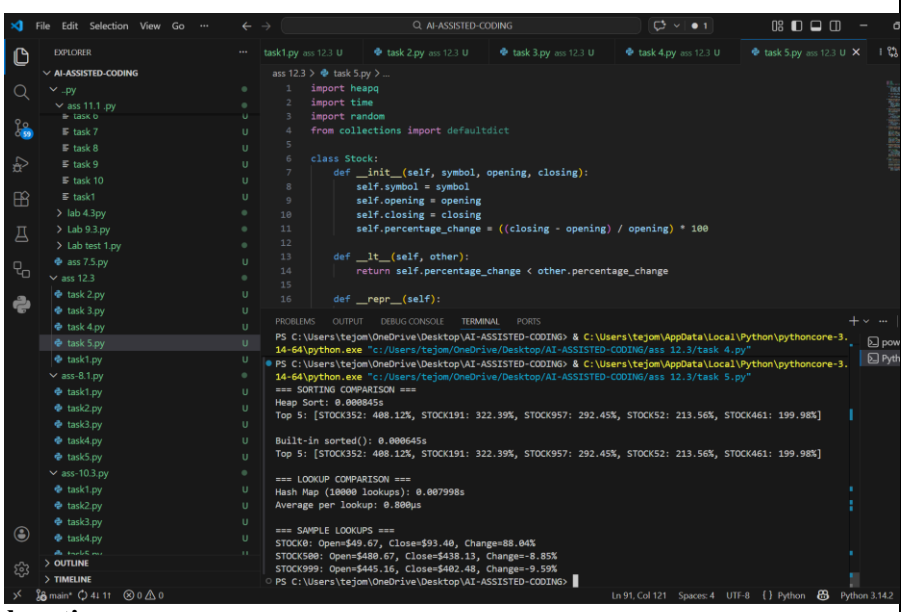
Search: Laptop found

Sorted by Price: Mouse, Keyboard, Laptop

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered **FinTech Lab** at SR University is building a tool for analyzing **stock price movements**. The requirement is to quickly **sort stocks by daily gain/loss** and search for specific stock symbols efficiently.



Explanation:

Stocks are sorted by percentage change using **Heap Sort** and searched using **Dictionary**.

	Output: Top Stock: TSLA +8.3% Search AAPL → Found (150 → 158)	